

1 Introduction

L'objectif de ce travail est de compléter l'initiation aux notions abordées pendant les séances de cours du module autour de l'intérêt d'utiliser des heuristiques et/ou des métaheuristiques lorsque le problème à résoudre est "*difficile*".

Pour cela vous allez travailler en équipe.

Une équipe est, par définition, composée de plusieurs individus. Il ne s'agit pas d'un travail individuel mais bien d'une collaboration qui doit notamment contribuer à renforcer vos compétences organisationnelles au sein d'un groupe.

L'organisation au sein de l'équipe n'est pas du ressort de l'équipe enseignante qui peut néanmoins intervenir en cas de difficultés ou problèmes. Il est **très important** que chaque membre de l'équipe puisse mettre en avant son rôle et ses contributions dans la réalisation du projet et son aboutissement. Ces éléments seront pris en compte dans la notation et pourront déboucher sur une note différente pour chacun.

D'un point de vue pratique, toutes les séances restantes du module seront consacrées à ce travail d'équipe.

2 Description du problème

On s'intéresse dans ce projet à l'utilisation d'heuristiques et de métaheuristiques pour résoudre le problème de planification des infirmières. Ce problème est connu dans la littérature sous la dénomination *nurse rostering*.

La planification des infirmières dans les structures hospitalières est un problème important et difficile, en raison du grand nombre de contraintes à satisfaire. De nombreux cas d'étude spécifiques peuvent être recensés dans la littérature, avec souvent des contraintes particulières liées à l'hôpital considéré ou encore au fonctionnement de ce dernier.

En 2014, Curtois et Qu [5] ont mis à disposition un nouveau jeu de données permettant de tester les nouvelles approches de résolution, avec notamment l'objectif de "*refléter les exigences du monde réel et les scénarios de planification tout en restant faciles à utiliser*". Ils fournissent une description détaillée des instances ainsi qu'une modélisation du problème considéré sous la forme d'un programme linéaire en nombres entiers. C'est ce problème que vous allez chercher à résoudre dans ce travail. Dans un premier temps nous réintroduisons l'essentiel des notations et le modèle d'après [5] (vous trouverez plus de détails dans les fichiers originaux, disponibles sur Moodle).

En résumé le problème consiste à construire un planning sur un horizon donné, en jour, chaque jour étant décomposé en un ensemble de postes (par exemple matin, après-midi, nuit), pour un ensemble de personnels. Chaque membre du personnel peut travailler durant un poste par jour, au maximum. Chaque poste requiert un certain nombre d'employés, et il n'est pas forcément possible de travailler sur n'importe quel poste après en avoir assuré un (par exemple il n'est pas

possible de travailler le matin si on a travailler la nuit précédente). D'autres contraintes sont associés aux postes et employés, nous y revenons un peu plus loin.

Le problème se caractérise par les paramètres et ensembles suivants :

- h , le nombre de jour de l'horizon de planification
- J , l'ensemble des jours concernés par la planification ($J = \{1, \dots, h\}$)
- W , l'ensemble des week-end concernés par la planification ($W = \{1, \dots, 7\}$ car on suppose ici que h est toujours un multiple de 7, et que le planning commence toujours un lundi)
- E , l'ensemble des employés
- P , l'ensemble des types de postes

Chaque type de poste $p \in P$ est caractérisé par :

- d_p , sa durée (en minutes)
- I_p , l'ensemble des types de postes qui ne peuvent pas être affectés immédiatement après le type de poste p
- u_{jp} , le nombre de personnel requis pour le type de poste p le jour $j \in J$.

Chaque employé $e \in E$ est caractérisé par :

- R_e , l'ensemble des jours où l'employé e ne travaille pas (repos demandé par exemple)
- $t_e^{\min}$, le temps total minimum de travail à faire par e
- $t_e^{\max}$, le temps total maximum de travail à faire par e
- $c_e^{\min}$, le nombre minimum de postes de travail consécutifs que l'employé e doit travailler
- $c_e^{\max}$, le nombre maximum de postes de travail consécutifs que l'employé e doit travailler
- $r_e^{\min}$, le nombre minimum de jour de repos consécutifs à affecter à l'employé e
- $w_e^{\max}$, le nombre maximum de week-ends que peut travailler l'employé e
- $m_{ep}^{\max}$, le nombre maximum de jours où e travaille sur le poste de type p ($p \in P$).

Chaque employé est aussi caractérisé par deux ensembles (éventuellement vides) de requêtes de type

- souhait d'être affecté à un type de poste particulier un jour de l'horizon de planification
- souhait de ne pas être affecté à un type de poste particulier un jour de l'horizon de planification.

La plupart des contraintes du problème sont dites *dures*, c'est-à-dire qu'elles doivent forcément être respectées dans une solution (sinon celle-ci n'est pas une solution faisable, elle ne peut pas être acceptée). Quelques contraintes sont dites *souples*, en particulier les contraintes de respect des requêtes d'affectations des personnels énoncées juste avant. Dans ce cas l'objectif est de les satisfaire, mais si une solution ne les satisfait pas complètement alors la solution est pénalisée. Les paramètres liés aux pénalités sont les suivantes :

- q_{ejp} , la pénalité si l'employé e n'est pas affecté au type de poste p le jour j alors qu'il le souhaitait
- p_{ejp} , la pénalité si l'employé e est affecté au type de poste p le jour j alors qu'il ne le souhaitait pas

Enfin, le fait de ne pas réussir à affecter autant de personnel que nécessaire (ou trop de personnel) pour chaque type de poste et chaque jour est aussi pénalisé dans la fonction objective du problème via :

- $v_{jp}^{\min}$, la pénalité si le nombre de personnels affectés au type de poste p le jour j est (strictement) inférieur à u_{jp}
- $v_{jp}^{\max}$, la pénalité si le nombre de personnels affectés au type de poste p le jour j est (strictement) supérieur à u_{jp} .

Le problème peut alors être modélisé en introduisant les variables de décision suivantes :

- x_{ejp} , égale à 1 si l'employé e est affecté au type de poste p le jour j , 0 sinon
- t_{ew} , égale à 1 si l'employé e travaille le week-end w , 0 sinon
- $\underline{y}_{jp}$, égale au manque de personnel affecté au type de poste p le jour j (un entier)
- $\overline{y}_{jp}$, égale à l'excédent de personnel affecté au type de poste p le jour j (un entier).

La liste des contraintes du problème :

1. Chaque employé ne peut être affecté qu'à un seul type de poste par jour, au plus :

$$\sum_{p \in P} x_{ejp} \leq 1, \forall e \in E, j \in J \quad (1)$$

2. Incompatibilité dans l'enchaînement de certains types de postes :

$$x_{ejp} + x_{e(j+1)p'} \leq 1, \forall e \in E, j \in \{1, \dots, h-1\}, p \in P, p' \in I_p \quad (2)$$

3. Chaque employé e est affecté à au plus $m_{ep}^{\max}$ fois le poste p :

$$\sum_{j \in J} x_{ejp} \leq m_{ep}^{\max}, \forall e \in E, p \in P \quad (3)$$

4. Chaque employé travaille un total total borné :

$$t_e^{\min} \leq \sum_{j \in J} \sum_{p \in P} d_p x_{ejp} \leq t_e^{\max} \quad (4)$$

5. Chaque employé travaille au plus $c_e^{\max}$ jours consécutifs :

$$\sum_{j=d}^{d+c_e^{\max}} \sum_{p \in P} x_{ejp} \leq c_e^{\max}, \forall e \in E, d \in \{1, \dots, h - c_e^{\max}\} \quad (5)$$

6. Chaque employé travaille au moins $c_e^{\min}$ jours consécutifs :

$$\sum_{p \in P} x_{edp} + \left(s - \sum_{j=d+1}^{d+s} x_{ejp} \right) + \sum_{p \in P} x_{e(d+s+1)p} > 0, \forall e \in E, s \in \{1, \dots, c_e^{\min} - 1\},$$

$$d \in \{1, \dots, h - (s+1)\} \quad (6)$$

7. Chaque employé a un nombre minimum consécutifs de jour de repos :

$$\left(1 - \sum_{p \in P} x_{edp} \right) + \sum_{j=d+1}^{d+s} \sum_{p \in P} x_{ejp} + \left(1 - \sum_{p \in P} x_{e(d+s+1)p} \right) > 0,$$

$$\forall e \in E, s \in \{1, \dots, r_e^{\min} - 1\}, d \in \{1, \dots, h - (s+1)\} \quad (7)$$

8. Chaque employé e ne doit pas travailler plus que $w_e^{\max}$ week-ends (un week-end est considéré travaillé si l'employé travaille au moins un des deux jours parmi samedi et dimanche) :

$$t_{ew} \leq \sum_{p \in P} x_{e(7w-1)p} + \sum_{p \in P} x_{e(7w)p} \leq 2t_{ew}, \forall e \in E, w \in W \quad (8-1)$$

$$\sum_{w \in W} t_{ew} \leq w_e^{\max}, \forall e \in E \quad (8-2)$$

9. Chaque employé ne travaille pas certains jours :

$$x_{ejp} = 0, \forall e \in E, j \in R_e, p \in P \quad (9)$$

10. Les contraintes de couvertures des postes :

$$\sum_{e \in E} x_{ejp} - \overline{y}_{jp} + \underline{y}_{jp} = u_{jp}, \forall j \in J, p \in P \quad (10)$$

Enfin, la fonction objective du problème :

$$\min \sum_{e \in E} \sum_{j \in J} \sum_{p \in P} q_{ejp}(1 - x_{ejp}) + \sum_{e \in E} \sum_{j \in J} \sum_{p \in P} p_{ejp} x_{ejp} + \sum_{j \in J} \sum_{p \in P} v_{jp}^{\min} \underline{y}_{jp} + \sum_{j \in J} \sum_{p \in P} v_{jp}^{\max} \overline{y}_{jp} \quad (11)$$

3 Travail attendu - tâches à réaliser

Dans ce projet vous devez étudier une ou plusieurs approches proposées pour aborder ce problème et implémenter une solution. Vous avez le droit de vous baser complètement sur une solution décrite dans un des articles fournis, ou de proposer une approche plus ou moins personnelle.

La validation de l'approche proposée se fera en utilisant les 24 instances générées par [5] et qui sont mises à disposition via Moodle. Deux formats d'instances sont fournis : une version texte et une version XML (extension `.ros`). Ce dernier format est utilisé par un outil de visualisation fourni (voir section 3.2).

Plusieurs choses sont attendues dans ce travail de groupe, certaines pouvant être réalisées en parallèle. A vous de voir comment vous vous organisez au sein de l'équipe, mais il est (fortement) conseillé que chacun aborde en partie tous les aspects du projet (en particulier si vous souhaitez que la notation soit homogène).

3.1 Bibliographie

Tout d'abord, un travail de **bibliographie** sur le sujet est évidemment indispensable. Celui-ci pourra être complété par une étude bibliographique sur les heuristiques et métaheuristiques pour compléter les notions abordées en cours, et vous aider à mieux appréhender les méthodes de résolution décrites dans les articles fournis.

—> Vous avez pour cela un ensemble de références disponibles sur la page Moodle du cours. Il est également autorisé de compléter ces ressources par d'autres éléments que vous trouveriez sur le web.

Cette étude bibliographique n'est pas à prendre à la légère et sera prise en compte dans l'évaluation de votre travail. Elle sera en effet présentée dans le rapport et il est important de ne pas oublier de citer toutes les références utilisées (voir section 4).

Au delà du document de base [5] qui présente en détails les instances et la formulation reprise dans la section 2, il y a trois articles dans lesquels les auteurs ont considéré les 24 instances qui nous concerne. Il semble donc évident que ces trois articles soient étudiés en détails [3, 7, 9].

D'autres références traitant du problème de planification des infirmières sont aussi mis à votre disposition [1, 2, 4, 6, 8]. Ces articles vous permettront de découvrir d'autres approches de résolution qui pourraient être utilisées, partiellement ou complètement, pour résoudre les 24 instances.

3.2 Optimisation

Pour résoudre ce problème, vous devez développer des métaheuristiques basée sur la **Variable Neighborhood Search (VNS)** et la **Recherche Taboo**. L'objectif de la métaheuristique utilisée est de chercher la meilleure solution possible, c'est-à-dire le planning conduisant à la plus petite pénalité. La représentation d'une solution et les structures de données utilisées sont libres.

Les éléments suivants sont obligatoires dans votre solution et doivent pouvoir être facilement identifiables :

- Une fonction de lecture des données à partir d'un fichier texte associé à chaque instance ;
- Une fonction permettant de construire un planning (i.e., une solution) faisable, c'est-à-dire respectant toutes les contraintes dures du problème ;

- Une fonction qui calcule et retourne la valeur d’une solution donnée ;
- Une fonction qui vérifie si une solution donnée est faisable ;
- Une fonction de sauvegarde d’une solution dans un fichier `.ros`, format XML utilisé par la solution de visualisation ;
- Un ensemble de fonctions permettant d’implémenter la métaheuristique.

Le langage de programmation utilisé est libre, et une partie des attendus réside dans la documentation de celui-ci.

Un outil de visualisation d’instances et de solutions est mis à disposition sur Moodle. Cet outil a été réalisé par la société *Staff Roster Solutions*. Vous pouvez ainsi l’utiliser pour mieux appréhender le contenu des instances et à quoi ressemble une solution. Vous pourrez également l’utiliser pour visualiser vos solutions (le format spécifique des fichiers pourra être déduit des exemples fournis).

4 Évaluation

La note finale de chacun sera déterminée à partir de plusieurs évaluations :

- Un rapport
- Le code source
- Les résultats obtenus

4.1 Rapport

Le rapport doit être un rapport de niveau d’élèves Master en fin d’étude. On entend par là qu’il doit être bien structuré, rédigé et présenté. La bibliographie sur le problème et sur les heuristiques et les métaheuristiques ne doit pas être négligée car elle doit en partie montrer ce que vous avez compris sur ces notions.

Attention : aucun plagiat ne sera toléré. Nous avons des outils à notre disposition via Moodle pour détecter ce genre de chose, en particulier pour les rapports. Le but de cette partie est de présenter, avec vos mots, les méthodes que vous avez retenu et leur fonctionnement.

Un soin particulier doit être apporté à la présentation de votre solution : structures de données utilisées, algorithmes mis en œuvre pour la résolution, etc.

La partie dédiée à la présentation des tests sur les 24 instances doit également être faite avec le plus grand soin. Vous devez montrer quelle a été votre démarche pour et quelles conclusions vous en tirez.

Finalement, il est fondamental que votre rapport mette clairement et précisément en évidence le rôle et les tâches réalisées par chacun. Un schéma illustrant aussi l’organisation et le déroulé en fonction des séances peut apporter un plus.

4.2 Code source et résultats obtenus

L’évaluation du code source est essentiellement réalisée sur la base de la structuration du code, la vérification qu’il correspond à la description dans le rapport, et qu’il permet effectivement d’obtenir les résultats rapportés.

Il est aussi attendu que le code soit bien documenté afin de pouvoir reproduire les tests.

4.3 Dépôt

Tous les éléments attendus (rapport et code source)devront être déposés sur Moodle avant la date limite, qui sera stricte et indiquée sur la page du cours.

Il est fortement conseillé de ne pas attendre la dernière minute pour déposer votre travail, afin de limiter les risques liés à un problème de réseau par exemple.

Le dépôt se fera sous la forme d'une archive au format `.zip`, `.rar` ou `.7z`.

Toutes les modalités seront reprises sur l'espace du cours Moodle et leur respect sera intégré dans la notation.

Références

- [1] Uwe AICKELIN et Kathryn A. DOWSLAND : An indirect genetic algorithm for a nurse-scheduling problem. *Computers & Operations Research*, 31(5):761–778, 2004.
- [2] Ruibin BAI, Edmund K. BURKE, Graham KENDALL, Jingpeng LI et Barry MCCOLLUM : A hybrid evolutionary approach to the nurse rostering problem. *IEEE Transactions on Evolutionary Computation*, 14(4):580–590, 2010.
- [3] Edmund K. BURKE et Tim CURTOIS : New approaches to nurse rostering benchmark instances. *European Journal of Operational Research*, 237(1):71–81, 2014.
- [4] Edmund K. BURKE, Timothy CURTOIS, Gerhard POST, Rong QU et Bart VELTMAN : A hybrid heuristic ordering and variable neighbourhood search for the nurse rostering problem. *European Journal of Operational Research*, 188(2):330–341, 2008.
- [5] Tim CURTOIS et Rong QU : Computational results on new staff scheduling benchmark instances. *ASAP Res. Group, School Comput. Sci., Univ. Nottingham, Nottingham, UK, Tech. Rep*, 2014.
- [6] Ghaith M. JARADAT, Anas AL-BADAREEN, Masri AYOB, Mutasem AL-SMADI, Ibrahim AL-MARASHDEH, Mahmoud ASH-SHUQRAN et Eyas AL-ODAT : Hybrid elitist-ant system for nurse-rostering problem. *Journal of King Saud University - Computer and Information Sciences*, 31(3):378–384, 2019.
- [7] Erfan RAHIMIAN, Kerem AKARTUNALI et John LEVINE : A hybrid integer programming and variable neighbourhood search algorithm to solve nurse rostering problems. *European Journal of Operational Research*, 258(2):411–423, 2017.
- [8] Ioannis P. SOLOS, Ioannis X. TASSOPOULOS et Grigorios N. BELIGIANNIS : A generic two-phase stochastic variable neighborhood approach for effectively solving the nurse rostering problem. *Algorithms*, 6(2):278–308, 2013.
- [9] Aykut Melih TURHAN et Bilge BILGEN : A hybrid fix-and-optimize and simulated annealing approaches for nurse rostering problem. *Computers & Industrial Engineering*, 145:106531, 2020.